

Informazione

Domanda 1 (la domanda comprende le sotto-domande da 1 a 3 del questionario)

Domanda 1

Risposta
errata

Punteggio
ottenuto 0,00
su 0,75

Rispondere YES/NO a ognuna delle domande seguenti

(motivazioni/spiegazioni saranno richieste in una successiva domanda)

In un sistema di memoria virtuale con un grado fisso di multiprogrammazione, il thrashing può verificarsi anche se la somma dei working set di tutti i processi è inferiore alla memoria fisica disponibile, ma l'algoritmo di sostituzione della pagina non è ottimizzato?

NO

✗

In un sistema con paginazione basata su inverted page table (IPT), è possibile che più pagine virtuali di processi diversi condividano una singola entry della tabella delle pagine da senza causare un conflitto nella traduzione degli indirizzi?

NO

✗

In un sistema di paginazione a richiesta che utilizza la sostituzione delle pagine locale (a un singolo processo), un processo con frequenza di page fault molto elevata può comunque causare una riduzione della memoria fisica disponibile per altri processi?

YES

✗

Your answer is incorrect.

La risposta corretta è: In un sistema di memoria virtuale con un grado fisso di multiprogrammazione, il thrashing può verificarsi anche se la somma dei working set di tutti i processi è inferiore alla memoria fisica disponibile, ma l'algoritmo di sostituzione della pagina non è ottimizzato? → YES, In un sistema con paginazione basata su inverted page table (IPT), è possibile che più pagine virtuali di processi diversi condividano una singola entry della tabella delle pagine da senza causare un conflitto nella traduzione degli indirizzi? → YES, In un sistema di paginazione a richiesta che utilizza la sostituzione delle pagine locale (a un singolo processo), un processo con frequenza di page fault molto elevata può comunque causare una riduzione della memoria fisica disponibile per altri processi? → NO

Domanda 2

Completo

Punteggio
max.: 0,75

Domanda 3

Completo

Punteggio
max.: 1,50

Si consideri un sistema a 64 bit con uno spazio di indirizzi virtuale di 2^{48} Byte, una dimensione di pagina di 4 KB e 16 GB di memoria fisica. Si supponga che il sistema utilizzi una inverted page table (IPT). Calcolare e spiegare quanto segue:

1. Il numero di bit necessari per il numero di frame fisico.
2. Il numero totale di entry nella inverted page table.
3. L'indirizzo virtuale assegnato a un processo è $0x00007FFFFFFF000$. Determinare l'indirizzo fisico se questo indirizzo virtuale è mappato al frame fisico 1024. Mostrare tutti i passaggi, incluso il calcolo del numero di pagina e dell'offset.

1) $16GB/4KB = 4M$ Frame disponibili. Per indirizzare 4M frame servono 22 bit

2) Numero totale di entry = Numero di frame disponibili (Indice della tabella indica il frame a cui verrà mappata la pagina) = 4M entry

3) $4K \cdot 1024 = 0x400000$. Numero di pagina = $0x7FFF\ FFFF\ F000 / 0x1000 = 0x7FFF\ FFFF\ F$ numero pagina.
Offset = 0 perchè le ultime 3 cifre ($16^3 = 4096 =$ una pagina) sono a 0.

Domanda 1 (la domanda comprende la sotto-domanda 4 del questionario)

Domanda 4

Completo

Punteggio
max.: 3,00

Si consideri un file system che utilizza l'**allocazione** di tipo **linked** per l'allocazione dei file. Il disco è diviso in blocchi di 4 KB ciascuno. Ogni blocco contiene un puntatore al blocco successivo, che occupa 4 byte del blocco. Un file richiede 5 MB di spazio di archiviazione.

- A. Calcolare il numero di blocchi del disco necessari per allocare l'intero file, incluso lo spazio necessario per i puntatori.
- B. Se i puntatori sono stati ridotti a 2 byte (presupponendo una capacità del disco inferiore), ricalcolare il numero di blocchi necessari. Questa modifica renderebbe l'archiviazione dei file più efficiente in termini di utilizzo dei blocchi?
- C. Confrontare il sovraccarico introdotto dai puntatori in entrambi i casi come percentuale dello spazio di archiviazione totale usato.

A) 1 blocco contiene 4092B di dato: $5\text{MB}/4092\text{B} = 1282$ Blocchi

B) 1 blocco conterrebbe 4094B di dato : $5\text{MB}/4094\text{B} = 1281$ Blocchi. Sarebbe più efficiente (di poco), con frammentazione interna minore

C) Sovraccarico caso 1 = $1282 \cdot 4\text{B} = 5128\text{B}$
Sovraccarico caso 2 = $1281 \cdot 2\text{B} = 2562\text{B}$

Domanda 3 (la domanda comprende le sotto-domande da 5 a 7 del questionario)

Domanda 5

Parzialmente
correttaPunteggio
ottenuto 0,40
su 0,60

A ognuna delle domande seguenti rispondere YES/NO

(in una domanda separata occorrerà fornire delle motivazioni/spiegazioni)

In un sistema con memoria virtuale basata su paginazione a richiesta e DMA, è possibile che si verifichi un page fault durante un trasferimento DMA, causando l'esito negativo del trasferimento se non vengono prese le dovute precauzioni?

Scegli... ▾

In un sistema con più dispositivi di I/O e una singola CPU, le operazioni di I/O possono essere eseguite in parallelo se la CPU è occupata nell'esecuzione di un processo?

YES



In un sistema che utilizza I/O basato su interrupt, è possibile che un interrupt venga perso se il controller di interrupt è occupato nell'elaborazione di un altro interrupt e il dispositivo che ha attivato il secondo interrupt non supporta l'accodamento degli interrupt?

YES



Your answer is partially correct.

Hai selezionato correttamente 2.

La risposta corretta è:

In un sistema con memoria virtuale basata su paginazione a richiesta e DMA, è possibile che si verifichi un page fault durante un trasferimento DMA, causando l'esito negativo del trasferimento se non vengono prese le dovute precauzioni? → YES,

In un sistema con più dispositivi di I/O e una singola CPU, le operazioni di I/O possono essere eseguite in parallelo se la CPU è occupata nell'esecuzione di un processo? → YES,

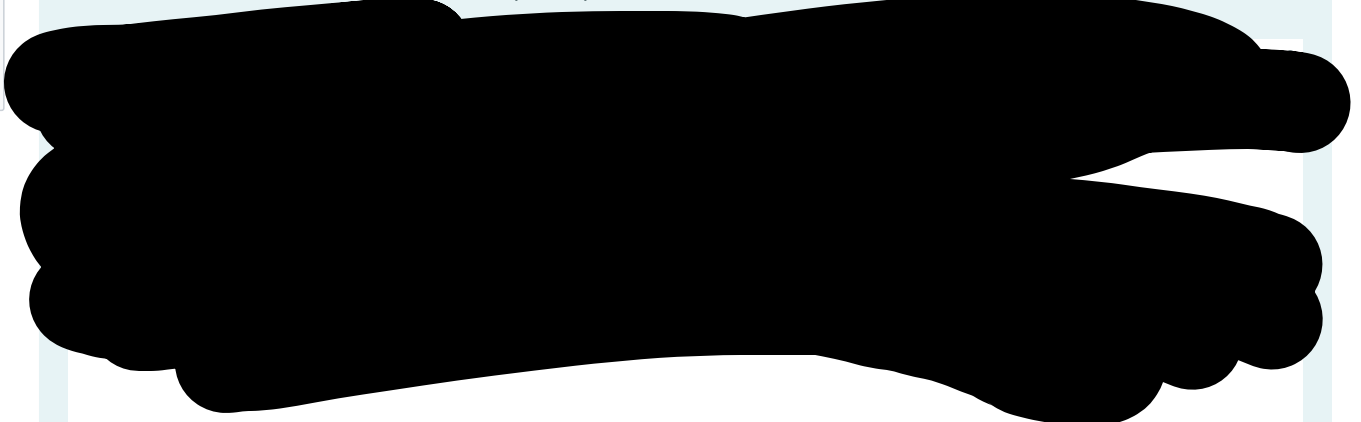
In un sistema che utilizza I/O basato su interrupt, è possibile che un interrupt venga perso se il controller di interrupt è occupato nell'elaborazione di un altro interrupt e il dispositivo che ha attivato il secondo interrupt non supporta l'accodamento degli interrupt? → YES

Domanda 6

Completo

Punteggio
max.: 0,90

Fornire una breve motivazione per le risposte precedenti (nello stesso ordine delle domande).



Domanda 7

Completo

Punteggio
max.: 1,50

D. Prendere in considerazione un'unità disco rigido (HDD) con le seguenti specifiche:

- Dimensione del settore: 512 byte
 - Numero di tracce per faccia: 5.000
 - Numero di settori per traccia: 300
 - Numero di piatti a doppia faccia: 6
 - Velocità di rotazione del piatto: 7.200 giri/min (giri al minuto)
1. Calcolare la velocità di trasferimento dati massima possibile in megabyte al secondo (MB/s), supponendo che sia possibile trasferire una traccia di dati per giro.
 2. Se il disco subisce un arresto anomalo della testina su un piatto, in che modo ciò influisce sulla capacità totale e sulla disponibilità dei dati supponendo che non siano in atto meccanismi RAID o di backup?
 3. Se il disco ha un tempo di ricerca medio di 4 ms e deve leggere un file da 1 GB suddiviso in 200 tracce non contigue, calcolare il tempo totale necessario per leggere il file. Includi il tempo per la ricerca, la latenza rotazionale e il trasferimento dei dati. Si supponga che la latenza rotazionale media sia di 4,165 ms e che sia possibile leggere una traccia per giro.

1) Ogni traccia contiene $512\text{B} \cdot 300 = 150\text{KB}$.

120 giri/s. $150\text{KB} \cdot 120 \text{ 1/s} = 18000 \text{ KB/s} = 17.58 \text{ MB/s}$

2) 1 piatto = $150\text{KB} \cdot 5000 = 732.4 \text{ MB}$

Capacità ridotta = $8.58 \text{ GB} - 732.4\text{MB} = 7.87 \text{ GB}$

3) 1 giro in $8.33\text{ms} = 1/120\text{s}$, latenza media = $200 \cdot (4.165 + 4 + 8.33) = 3299\text{ms} = 3.3\text{s}$

Il file di 1 GB non può stare in 200 tracce, però. O ne leggiamo solo 1/3 oppure

Se consideriamo che 1GB può stare in $6991/12 = 583$ tracce nelle 12 facce => 8.25s

Informazione

Domanda 4 (la domanda comprende le sotto-domande da 8 a 9 del questionario)

Si consideri un sistema oS161, in esecuzione su un simulatore MIPS sys161 con 8 MB di memoria RAM.

Domanda 8

Parzialmente corretta

Punteggio ottenuto 1,00 su 1,50

La memoria fisica in dumbvm viene allocata chiamando la funzione ram_stealmem. Per ciascuna delle seguenti frasi, rispondere vero/falso (true/false: non è necessaria alcuna spiegazione)

- Sebbene la soluzione proposta per supportare la deallocazione della memoria (freeppages) sia basata su una bitmap, potrebbe essere utilizzata anche una lista concatenata

TRUE



- La funzione ram_stealmem alloca in modo incrementale la memoria a partire dal primo indirizzo disponibile dopo aver fatto load del kernel

TRUE



- Quando si utilizza una bitmap per tenere traccia della RAM allocata/libera, è necessario un secondo array che memorizzi le dimensioni delle partizioni allocate per l'allocazione sia della memoria dinamica del kernel che dei processi utente

FALSE



- Lo stack di un processo utente non viene allocato (in dumbvm) chiamando ram_stealmem, perché la dimensione dello stack è una costante e lo stack si trova sempre agli stessi indirizzi logici.

FALSE



- Lo schema di allocazione utilizzato da ram_stealmem non consentirà l'intercalare partizioni contigue del kernel e dei processi user (in altre parole, le partizioni del kernel e le partizioni user non saranno mai interlacciate)

TRUE



- La funzione ram_stealmem viene chiamata per l'allocazione della memoria sia del kernel (memoria dinamica) che dei processi user

FALSE



Your answer is partially correct.

Hai selezionato correttamente 4.

La risposta corretta è:

- Sebbene la soluzione proposta per supportare la deallocazione della memoria (freepages) sia basata su una bitmap, potrebbe essere utilizzata anche una lista concatenata → TRUE,
- La funzione ram_stealmem alloca in modo incrementale la memoria a partire dal primo indirizzo disponibile dopo aver fatto load del kernel → TRUE,
- Quando si utilizza una bitmap per tenere traccia della RAM allocata/libera, è necessario un secondo array che memorizzi le dimensioni delle partizioni allocate per l'allocazione sia della memoria dinamica del kernel che dei processi utente → FALSE,
- Lo stack di un processo utente non viene allocato (in dumbvm) chiamando ram_stealmem, perché la dimensione dello stack è una costante e lo stack si trova sempre agli stessi indirizzi logici. → FALSE,
- Lo schema di allocazione utilizzato da ram_stealmem non consentirà l'intercalare partizioni contigue del kernel e dei processi user (in altre parole, le partizioni del kernel e le partizioni user non saranno mai interlacciate) → FALSE,
- La funzione ram_stealmem viene chiamata per l'allocazione della memoria sia del kernel (memoria dinamica) che dei processi user → TRUE

Domanda 9

Completo

Punteggio
max.: 1,50

È noto che uno spazio di indirizzamento utente è caratterizzato da `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2`, `as->as_stackpbase`, aventi i seguenti valori: 0x200000, 0x300000, 0x3000, 0x7000, 3, 4, 0x400000. Le pagine in OS161 hanno una dimensione di 4 KB. È anche noto che a uno stack user vengono allocate 18 pagine in dumbvm.

Dati i seguenti indirizzi logici (potrebbero essere sia user che kernel) convertirli nei relativi indirizzi fisici.

- 0x8110

- 0x6500

- 0x7FFFE010

- 0x805000B0

- 0x8110 -> 0x201 110

- 0x6500 -> Fuori dallo spazio di indirizzamento user , 0x8000 6500

- 0x7FFFE010 -> 0x3FE 010 stack, sta alla fine dell'indirizzamento logico user (Entro i -0x13000)

- 0x805000B0 -> 0x500 0B0

Domanda 5 (la domanda comprende le sotto-domande da 10 a 11 del questionario)

Domanda 10

Parzialmente corretta

Punteggio ottenuto 0,75 su 1,50

Considerare la funzione `sys_getpid` (prototipo sotto)

`pid_t sys_getpid(void);`

Implementato in Lab4. Per ciascuna delle seguenti frasi, rispondere vero/falso (true/false: non è necessaria alcuna spiegazione)

- La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall`

FALSE

✗

- La funzione restituisce il pid del processo corrente

TRUE

✓

- La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa

TRUE

✗

La funzione viene chiamata da `sys__exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione

FALSE

✓

La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione)

FALSE

✓

- La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente

TRUE

✗

Your answer is partially correct.

Hai selezionato correttamente 3.

La risposta corretta è:

- La funzione è necessaria per implementare la chiamata di sistema `getpid`, e sarà chiamata dal kernel, nella funzione `syscall` → TRUE,

- La funzione restituisce il pid del processo corrente → TRUE,

- La funzione è necessaria per implementare `sys_waitpid`, perché è necessario il pid del processo in attesa → FALSE,

La funzione viene chiamata da `sys__exit`, in modo da conoscere il pid del processo a cui inviare una segnalazione → FALSE,

La funzione viene chiamata per ottenere il pid di un altro processo (quello di cui attendere la terminazione) → FALSE,

- La funzione è necessaria per implementare la chiamata di sistema `getpid` e verrà chiamata direttamente da un processo utente → FALSE

Domanda 11

Completo

B) Viene riportata una versione della funzione `proc_wait`, fornita all'interno della soluzione proposta nel laboratorio 4. La funzione contiene 3 errori. Trovarli e, per ognuno di essi, fornire una breve spiegazione (perché sono errori?)

```
int proc_wait(struct proc *proc)
{
    int return_status;

    KASSERT(proc != NULL);
    KASSERT(proc != kproc);

    cv_wait(proc->p_cv, proc->p_lock);
    lock_acquire(proc->p_lock);
    return_status = proc->p_status;
    if (return_status < 0)
        return return_status;
    lock_release(proc->p_lock);
    proc_destroy(curproc);
    return return_status;
}
```

	Errore	Spiegare
1		
2		
3		

Errors

